Input Collector - Arquitectura y Documentación Técnica

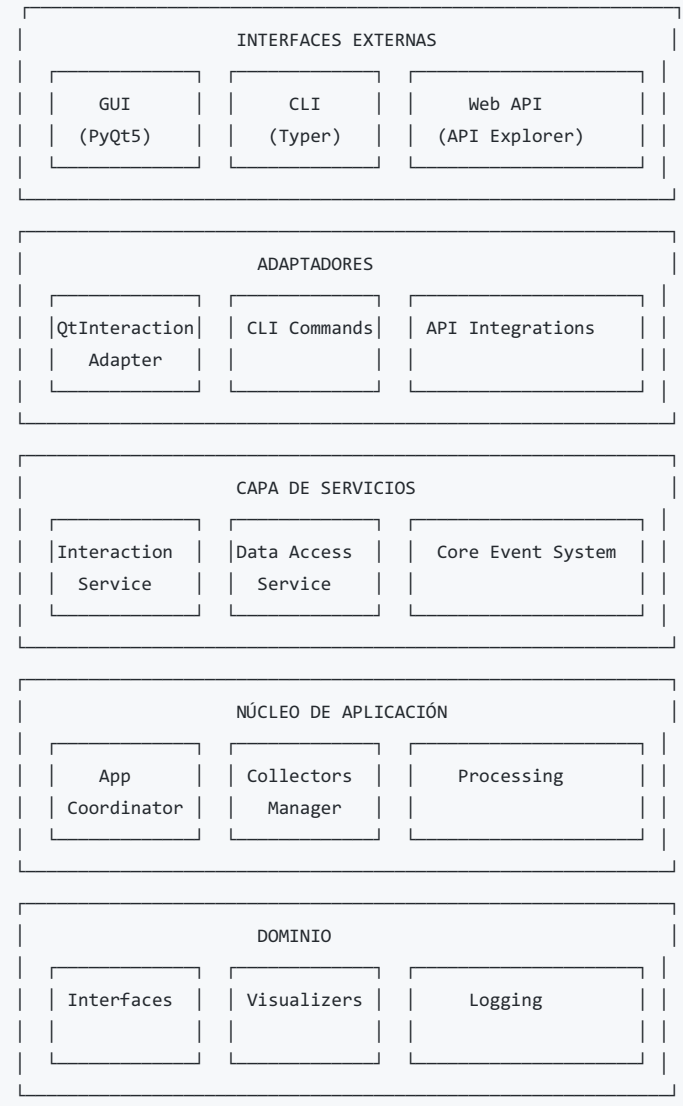
Resumen

Input Collector es una aplicación integral diseñada para capturar, procesar, visualizar y registrar eventos de entrada en tiempo real desde múltiples dispositivos (teclado, ratón, controladores PS4). La aplicación implementa una arquitectura modular basada en eventos que soporta múltiples interfaces de usuario (GUI, CLI, Web API) y proporciona capacidades extensibles para diferentes tipos de colectores y modos de visualización.

Arquitectura General

Patrón Arquitectónico Principal

La aplicación sigue una **arquitectura hexagonal (Ports and Adapters)** combinada con patrones **Mediator** y **Observer**, organizando el código en capas bien definidas:



Submódulos y Componentes

1. Core Event System (`core/`)

Propósito: Sistema de eventos centralizado y framework-independiente para comunicación desacoplada.

Componentes:

- **`EventDispatcher`** : Singleton que gestiona suscripciones y dispatch de eventos
- **`EventType`** : Enumeración de tipos de eventos del sistema
- **Funciones utilitarias**: API simplificada para uso común

Características:

- Patrón Singleton thread-safe
- Gestión de suscriptores basada en sets
- Almacenamiento del último evento para sincronización tardía
- Aislamiento de errores en callbacks
- Diseño framework-independiente

```
# Ejemplo de uso
from core import dispatch, subscribe, EventType

def on_input_event(data):
    print(f"Received input: {data}")

subscribe(EventType.INPUT, on_input_event)
dispatch(EventType.INPUT, {"key": "A", "timestamp": time.time()})
```

2. Interfaces (`interfaces/`)

Propósito: Definición de contratos y estructuras de datos fundamentales.

Componentes:

- **`InputData`**: Dataclass principal para eventos de entrada
- **`AbstractInputCollector`**: Interfaz para colectores de entrada
- **`AbstractVisualizer`**: Interfaz para visualizadores
- **`AbstractVisualizerProvider/Controller`**: Protocolos para gestión de UI

Características:

- Dataclass comprehensiva con 25+ campos para diferentes tipos de entrada
- Interfaces abstractas que garantizan contratos consistentes
- Protocolos Python para tipado estructural
- Método `to_dict()` para serialización

3. Collectors (`collectors/`)

Propósito: Captura de eventos de entrada desde diferentes dispositivos.

Componentes:

- **`CollectorManager`**: Gestor central de todos los colectores
- **`KeyboardMouseCollector`**: Colector para teclado y ratón
- **`PS4ControllerCollector`**: Colector para mandos PS4 (opcional con SDL2)

Características:

- Patrón Strategy para diferentes tipos de colectores
- Múltiples modos de seguimiento de ratón (delta, ángulo, ejes, joystick)
- Detección automática de dispositivos
- Gestión de estado en tiempo real
- Callbacks asíncronos para eventos

```
# Modos de tracking de ratón
TRACKING_MODES = {
    "delta": "Raw movement deltas",
    "angle": "Direction and angle",
    "axis": "Cardinal directions (N/S/E/W)",
    "joystick": "Direction and speed"
}
```

4. Processing (`processing/`)

Propósito: Procesamiento, formateo y transformación de eventos.

Componentes:

- **`EventProcessor`**: Procesador central de eventos

Características:

- Detección de eventos especiales (F9 para toggle recording)

- Formateo específico por tipo de colector
- Filtrado de eventos para logging
- Generación de texto de display
- Manejo de múltiples modos de tracking

5. Services (`services/`)

Propósito: Capa de servicios singleton para lógica de negocio.

Componentes:

- `InteractionService`: Servicio para control de aplicación
- `DataAccessService`: Servicio para acceso a datos en tiempo real

Características:

- Patrón Singleton thread-safe
- Registro de componentes para acoplamiento débil
- Categorización de eventos por tipo
- Sistema de callbacks para patrones reactivos
- Límites de historial para gestión de memoria

6. Visualizers (`visualizers/`)

Propósito: Componentes de visualización en tiempo real para diferentes tipos de entrada.

Componentes:

- `BaseVisualizer`: Clase base común
- `KeyboardVisualizer`: Teclado virtual con highlighting
- `MouseVisualizer`: Visualización estilo joystick para ratón
- `MouseButtonsSelector`: Selector de botones de ratón
- `PS4ControllerVisualizer`: Visualizador de mando PS4 con overlays de imagen

Características:

- Herencia de BaseVisualizer para comportamiento consistente
- Modo de edición para selección de teclas/botones
- Múltiples modos de visualización (imagen/dibujado)
- Actualización en tiempo real de estados
- Soporte para diferentes capacidades de colectores

7. Logging (`logging/`)

Propósito: Sistema de registro a múltiples formatos.

Componentes:

- `LoggingManager`: Coordinador central de logging
- `JSONLogger`: Logger para formato JSON estructurado
- `CSVLogger`: Logger para formato CSV tabular

Características:

- Logging simultáneo a múltiples formatos
- Generación dinámica de headers CSV
- Formateo específico por tipo de colector
- Señales Qt para comunicación thread-safe
- Gestión automática de archivos

8. UI (`ui/`)

Propósito: Gestión de interfaz de usuario y coordinación de visualizadores.

Componentes:

- `UIManager`: Gestor principal de UI con tema oscuro
- `VisualizerManager`: Coordinador de visualizadores
- `KeyboardEventFilter`: Filtro de eventos para modo de edición

Características:

- Tema oscuro consistente
- Layout responsive
- Gestión de estado compleja (recording, edit mode, collector type)

- Implementación implícita de protocolos para evitar conflictos de metaclass
- Señales Qt para comunicación asíncrona

9. App Coordinator (`app/`)

Propósito: Mediador central que implementa el patrón Mediator.

Componentes:

- **`AppCoordinator`**: Coordinador principal de la aplicación

Características:

- Patrón Mediator centralizado
- Gestión de estado de alto nivel
- Coordinación entre collectors, UI, logging y processing
- Procesamiento thread-safe via señales Qt
- Dispatch a sistema de eventos core para consumidores externos

10. Adapters (`adapters/`)

Propósito: Adaptadores thread-safe entre frameworks y servicios core.

Componentes:

- **`QtInteractionAdapter`**: Adaptador para operaciones UI thread-safe

Características:

- Patrón Adapter para aislamiento de framework
- Ejecución thread-safe con timeouts
- API comprehensiva para integración externa
- Manejo de errores robusto
- Tipado genérico para seguridad de tipos

11. CLI (`cli/`)

Propósito: Interfaz de línea de comandos completa.

Componentes:

- **`CLIApp`**: Aplicación principal CLI con Typer
- **`CLIShell`**: Shell interactivo estilo REPL
- **`CLIConfig`**: Gestión de configuración y estado compartido
- **`HeadlessInputCollector`**: Stack de aplicación para modo headless
- **Commands**: Módulos de comandos especializados

Características:

- Soporte dual (standalone/concurrent con GUI)
- Shell interactivo con comandos built-in
- Modo headless completo
- Monitoreo en tiempo real cross-platform
- Framework Rich para output mejorado

12. API Integration (archivos raíz)

Propósito: Exposición de APIs via web para acceso remoto.

Componentes:

- **`InputCollectorAPI`**: API estática para control de aplicación
- **`InputDataAPI`**: API estática para acceso a datos
- **`InputCollectorApiIntegration`**: Integración web para datos
- **`InteractionApiIntegration`**: Integración web para control

Características:

- Fachadas estáticas sobre capa de servicios
- Integración con API Explorer para web
- WebSocket para eventos en tiempo real
- Fallbacks para operación sin web server
- Tracking de estado PS4 especializado

Patrones Arquitectónicos Implementados

1. Patrón Mediator

- `AppCoordinator` actúa como mediador central
- Reduce acoplamiento entre componentes principales
- Centraliza lógica de coordinación compleja

2. Patrón Singleton

- `EventDispatcher`, `InteractionService`, `DataAccessService`
- Thread-safe con doble-checked locking
- Estado global consistente

3. Patrón Observer

- Sistema de eventos core
- Callbacks en servicios
- Señales Qt para UI

4. Patrón Strategy

- Diferentes colectores intercambiables
- Múltiples modos de tracking de ratón
- Diferentes visualizadores por tipo de entrada

5. Patrón Adapter

- `QtInteractionAdapter` para aislamiento de framework
- Adaptación entre Qt y servicios core
- Thread-safety transparente

6. Patrón Facade

- APIs estáticas como fachadas sobre servicios
- Simplificación de interfaces complejas
- Ocultación de implementación singleton

7. Patrón Factory

- Creación de colectores
- Inicialización de componentes UI
- Gestión de visualizadores

Características Clave del Sistema

1. Arquitectura Dirigida por Eventos

- Sistema de eventos core framework-independiente
- Comunicación desacoplada entre componentes
- Extensibilidad para nuevos tipos de eventos

2. Thread Safety Comprehensivo

- Múltiples niveles de locks granulares
- Ejecución thread-safe para operaciones UI
- Aislamiento de errores en callbacks

3. Multi-Interfaz

- **GUI**: Aplicación PyQt5 completa
- **CLI**: Shell interactivo y comandos batch
- **Web API**: Control remoto via HTTP/WebSocket

4. Extensibilidad

- Interfaces abstractas para nuevos colectores
- Sistema de visualizadores pluggable
- Arquitectura modular bien definida

5. Configurabilidad Avanzada

- Múltiples modos de tracking de ratón
- Configuración dinámica de teclas tracked
- Modo de edición para personalización

6. Logging Robusto

- Múltiples formatos simultáneos
- Headers dinámicos para CSV
- Formateo específico por colector

7. Visualización en Tiempo Real

- Updates de baja latencia
- Múltiples modos de display
- Overlays de imagen para PS4

8. Gestión de Estado Sofisticada

- Estado centralizado en servicios
- Sincronización automática
- Rollback y recuperación

Flujo de Datos Principal

```
1. Input Device → Collector → InputData
2. InputData → EventProcessor → ProcessingResult
3. ProcessingResult → AppCoordinator → Coordination
4. Coordination → {UI Update, Logging, Core Events}
5. Core Events → {API Broadcasts, CLI Monitoring}
```

Casos de Uso Principales

1. Desarrollo de Juegos

- Análisis de patrones de entrada de jugadores
- Testing de controles y responsividad
- Métricas de rendimiento de input

2. Investigación UX/UI

- Análisis de interacciones de usuario
- Patrones de uso de ratón y teclado
- Métricas de accesibilidad

3. Automatización y Testing

- Grabación de macros complejas
- Testing automatizado de interfaces
- Validación de input handling

4. Monitoreo de Sistema

- Análisis de uso de periféricos
- Detección de patrones anómalos
- Métricas de productividad

Tecnologías y Dependencias

Core

- **Python 3.8+**: Lenguaje principal
- **PyQt5**: Framework GUI
- **dataclasses**: Estructuras de datos
- **threading**: Concurrencia

CLI

- **Typer**: Framework CLI moderno
- **Rich**: Output mejorado
- **Cross-platform**: soporte Windows/Unix

Input Capture

- **pynput**: Captura teclado/ratón
- **pysdl2** (opcional): SDL2 para PS4
- **Platform-specific**: APIs nativas

Web API

- **API Explorer**: Framework web para APIs
- **WebSocket**: Comunicación en tiempo real
- **HTTP**: APIs REST